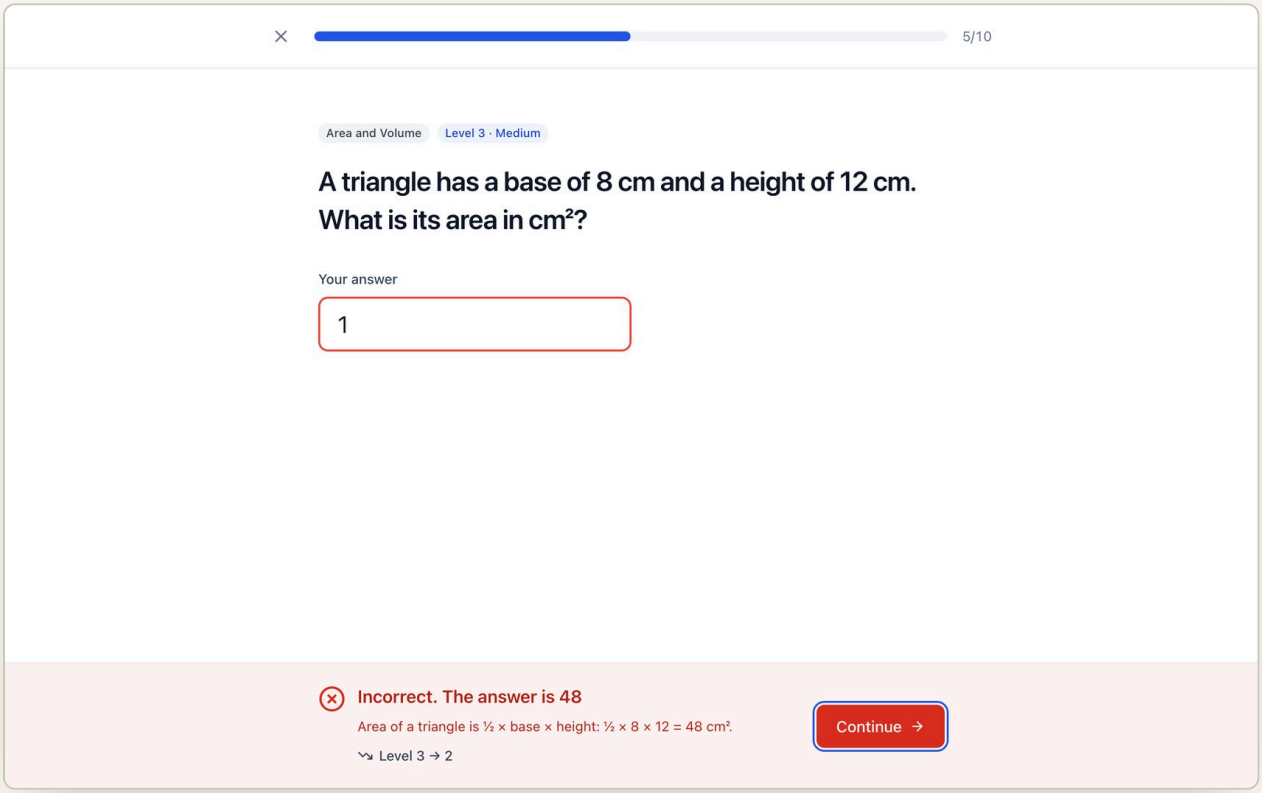
LearnLoop

How it works, from the browser to the database

A walk through the whole system: what a student, a teacher and an admin see, and what happens behind each click.



Pass 1 of 4
SRS weeks 1 and 2

October 2026
Version 1.0

154 tests
passing

localhost:8080
Docker stack

CONTENTS

What's inside

01 What LearnLoop is

02 The big picture (architecture)

03 One request, start to end

04 Logging in and staying logged in

05 The student journey

06 How adaptive practice works

07 The teacher journey

08 The admin journey and the audit log

09 The database

10 Security, layer by layer

11 Testing and CI/CD

12 Running it and the design system

13 Scaling, and what comes next

01 · What LearnLoop is

An adaptive maths practice platform for grades 6 to 8. Students practise; the questions get harder when they do well and easier when they struggle. Teachers see where the class is stuck. An admin manages accounts and can audit everything.

The three roles

ROLE	GOAL
Student	Practise, get instant feedback, see progress, finish set homework
Teacher	Run classes, build quizzes, assign work, spot students who need help
Admin	Manage users and roles, read the audit log

Where it stands

PHASE	STATUS
Week 1: foundation	Done
Week 2: core product	Done
Week 3: AI + MCP server	Next
Week 4: hardening, deploy	Planned

Reading tip. Each section says *what you see*, then *what happens behind it*, then *why it was built that way*. File names are given so you can open the code next to this guide.

Where to read more. `README.md` (overview), `SECURITY.md` (controls), `docs/adr/` (why decisions were made), `docs/design.md` (visual rules), `CLAUDE.md` (project memory and the scale rules).

Explore the API yourself

Every endpoint is listed, with its allowed roles and input rules, at `http://localhost:8080/api/docs`. The page is generated from the same declarations the server uses to check requests, so it can never drift out of date. There are 30 endpoints.

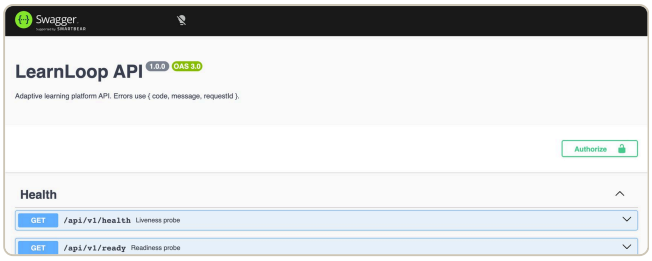
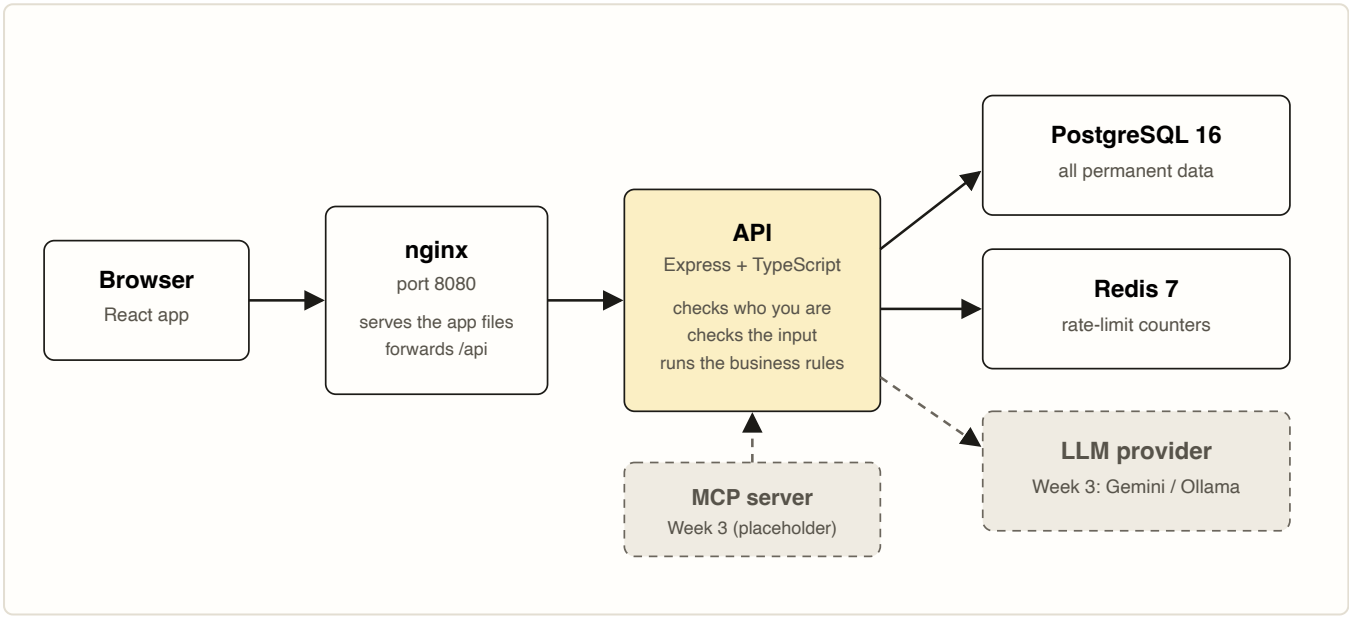


Fig 1 · The API documentation page.

02 · ARCHITECTURE

The big picture

Five containers run together. The browser only ever talks to one address, `localhost:8080`.



What each part does

PART	JOB	WHERE
Web app	The screens. Built with React, TypeScript and Vite, styled with Tailwind. It never touches the database; it only calls the API.	`apps/web`
nginx	Serves the built web files and forwards anything starting with `/api` to the API. This makes the app and API share one origin, which lets the strict cookies work.	`infra/nginx.conf`
API	The only thing that can read or write data. Every rule (who may do what, what input is valid, how difficulty adapts) lives here.	`apps/api`
PostgreSQL	Stores users, classes, questions, answers, scores and the audit log. Accessed through Prisma.	`apps/api/prisma`
Redis	Counts login attempts per IP and per account so brute-force guessing is blocked, even with several API servers.	`middleware/rateLimit.ts`

Shared package	The input rules (Zod schemas) and response shapes, used by *both* the web app and the API so they can never disagree.	`packages/shared`

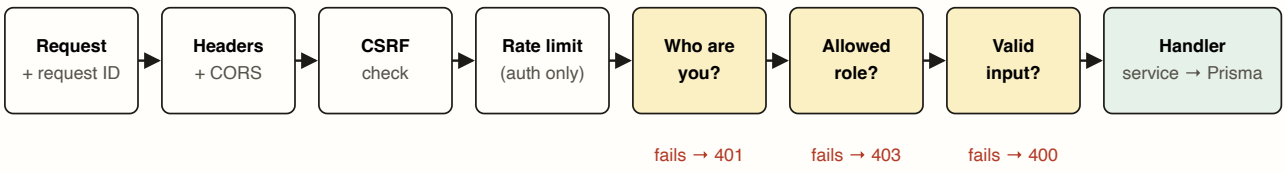
Why one API for everything? The web app and (in Week 3) the AI-facing MCP server both call the same API. So one set of permission checks, validation and audit logging protects every way into the data. There is no back door.

The repository: `apps/web` (front end) · `apps/api` (API, schema, seed, tests) · `packages/shared` (schemas and types) · `infra` (Docker, nginx) · `apps/mcp-server` (Week 3 placeholder) · `docs` (decisions, design, this guide).

03 · ONE REQUEST, START TO END

What happens when you click something

Every action, from "Check answer" to "Approve question", travels the same path. Learn this path once and the rest of the system makes sense.



Any failure becomes one tidy error shape: { code, message, requestId }. No stack traces ever leave the server.

Follow one click: "Check answer" on a practice question

1. **The browser** sends `POST /api/v1/practice/sessions/:id/answer` with the answer, the question ID, and how long the student took. It adds the access token and the CSRF token as headers, and nginx forwards the request to the API.
2. **The API's first layers** stamp a request ID on the log line, add security headers, check the origin is allowed, and verify the CSRF token matches the cookie.
3. **Authentication** checks the token's signature and expiry, then re-reads the user from the database. A disabled account or changed role takes effect on the very next click.
4. **The role check** confirms this route allows students. A teacher calling it gets a 403.
5. **Zod validation** confirms the body has exactly the expected fields. Unknown fields are rejected.
6. **The handler** (`services/practice.ts`) checks that this session belongs to *this* student, compares the answer on the server, adapts the difficulty, saves the attempt, updates mastery, and returns the verdict with the explanation.
7. **The browser** shows the green or red marking, the explanation, and any level change.

The key design idea. Each route is declared once (`ApiModule.route` in `lib/route.ts`) with its path, allowed roles and input shape. That one declaration drives the login check, the role check, validation *and* the API docs, so a route cannot exist without saying who may use it.

Logging in and staying logged in

The login page shows what the product looks like on the right, and three demo buttons sign you in as each role.

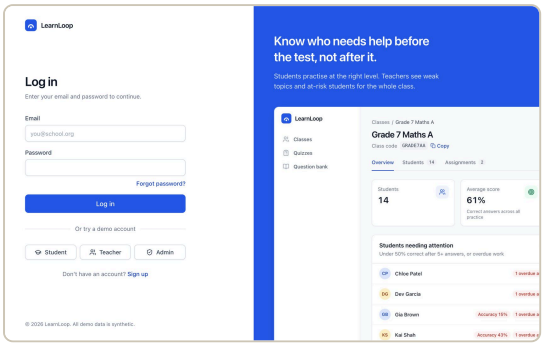


Fig 2 · The login page. The right side is a real screenshot of the product; "Or try a demo account" signs in instantly.

Step by step

1. **Rate limit.** Redis counts attempts per IP and per email. Too many and the API answers 429 with a `Retry-After` header.
2. **Find the user and check the password.** Passwords are stored as Argon2id hashes. If the email doesn't exist, the server still hashes against a dummy value, so the response time doesn't reveal which emails are registered.
3. **Issue three things.** A short-lived *access token* (a signed JWT, valid 15 minutes), a long-lived *refresh token* (random, 7 days) and a *CSRF token*.
4. **Store safely.** The refresh token goes into an HttpOnly, SameSite=Strict cookie that JavaScript cannot read. The database keeps only its SHA-256 hash. The access token lives in the page's memory only, never in localStorage.
5. **Audit.** Success and failure are both written to the audit log.

Staying logged in

SITUATION	WHAT HAPPENS
Access token expires (15 min)	The web client quietly calls `/auth/refresh` using the cookie, gets a new access token, and retries the failed request. The user notices nothing.
Page reload	Memory is empty, so the app calls `/auth/refresh` first. If the cookie is valid, the session is restored.
Every refresh	The refresh token is **rotated**: the old one is marked used, a new one issued. All tokens from one login share a "family".
An old refresh token is replayed	That means it was stolen. The **whole family is revoked**, the attempt is audited, and everyone using that login is signed out.
Logout	The family is revoked and the cookies are cleared.
Admin disables an account	All its refresh tokens are revoked, and the per-request check rejects its access token immediately.

> **Why this design?** A stolen access token is only useful for 15 minutes. A stolen refresh token can be used once; after that, rotation exposes the theft. Keeping the access token out of storage means injected scripts can't lift it.

Password reset works the same careful way: a single-use link, valid 30 minutes, stored only as a hash, and the response is identical whether or not the email exists.

What a student does

From login to the first question takes two clicks. The home page always offers exactly one best next action.

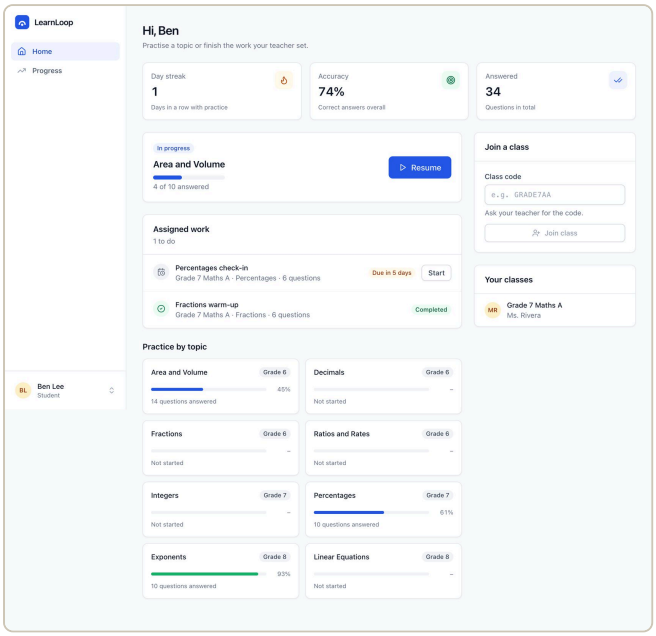


Fig 3 · Home: your stats, what to do next, assigned work, topics and classes.

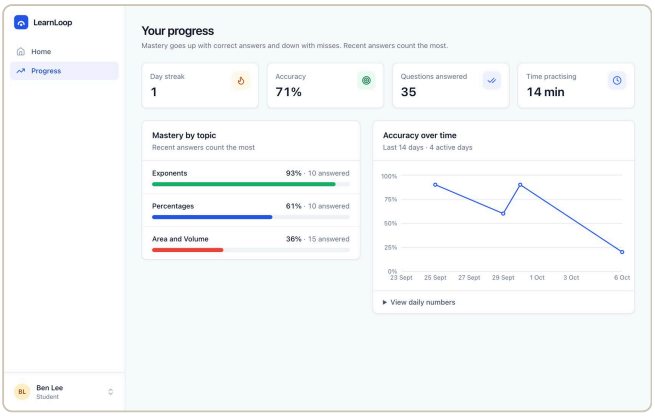


Fig 4 · Progress: four stats, mastery per topic and a 14-day accuracy chart.

What each part does

- **The summary cards** show the day streak, overall accuracy and total questions answered.
- **The "In progress" or "Suggested" card** resumes an unfinished session if there is one; otherwise it suggests the student's weakest topic (or one they haven't tried).
- **Assigned work** lists assignments with due dates. Overdue work is flagged in red, and finished work is marked "Completed".
- **Topics** shows mastery per topic. Clicking one starts a 10-question session.
- **Join a class** takes the teacher's code (for example `GRADE7AA`). Wrong codes and archived classes look the same ("not found"), and guessing is rate limited.
- **Progress** (Fig 4) shows streak, overall accuracy, questions answered, time spent, a mastery bar per topic and an accuracy line for the last 14 days. The chart also has a "View daily numbers" table for screen readers.

Practising a question

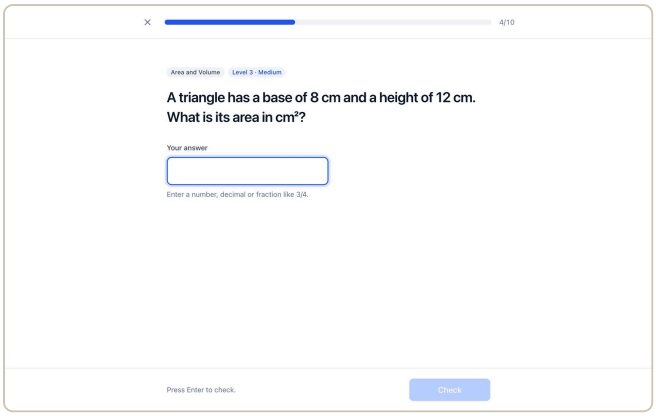


Fig 5 · The full-screen lesson page: close button and progress bar on top, one question, and a fixed bottom bar.

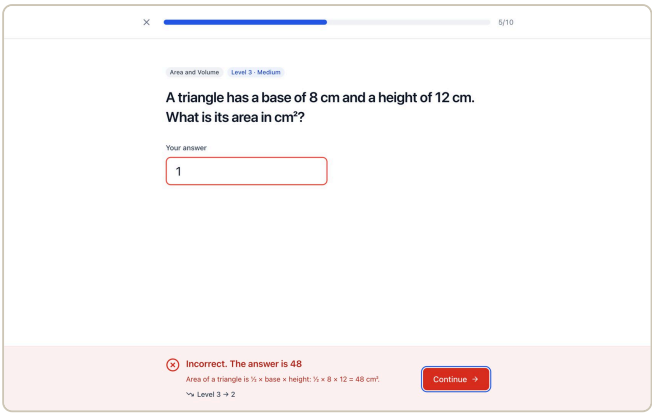
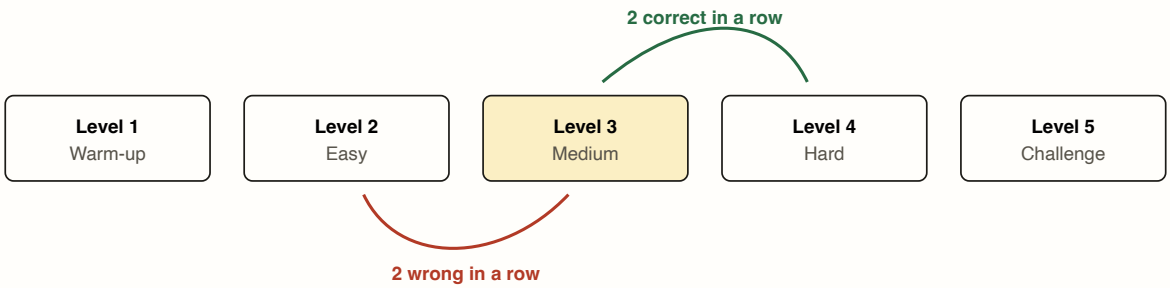


Fig 6 · After a wrong answer the bottom bar turns red with the correct answer, the worked explanation and a Continue button.

- The lesson page has no sidebar, so nothing distracts from the question. The **X** saves and leaves, and the bar on top fills as questions are answered.
- The bottom bar holds **Check**. After checking, it turns green or red, shows the answer and explanation, and offers **Continue**. If the level changes, it says so (for example "Level 4 → 3").
- Multiple-choice questions can be answered with the keys `A` to `D`, and `Enter` submits.
- The marking is announced to screen readers, and focus moves to "Continue" automatically.
- Leaving is safe: the session is stored on the server, so closing the tab and returning shows the same question.

How the difficulty adapts

The rule is deliberately simple so that a student can understand it: **two right in a row, step up; two wrong in a row, ease off.**



The pieces

PIECE	HOW IT WORKS	FILE
Starting level	From the student's current mastery of the topic: none or below 40% starts at 2; below 70% at 3; above that at 4.	`services/adaptive.ts`
Streak counter	Stored on the session: +n after n right in a row, –n after n wrong. At ±2 the level moves one step and the streak resets, so "two in a row" always means two *at the current level*. Level stays within 1 to 5.	`nextDifficulty()`
Picking a question	A random approved question in the topic, near the target level, that the student hasn't seen in this session. If none is left the band widens (±1, ±2, ±4).	`services/practice.ts`
Mastery score	A running average for the first 5 answers, then a weighted average where recent answers count 20%. One bad answer can't wreck an established score.	`updateMastery()`
Checking the answer	On the server only. Numeric answers compare by value, so `3/4`, `0.75` and `1,250` all work. The browser never receives the answer before submitting.	`isAnswerCorrect()`
Assigned quizzes	Do *not* adapt. They are a fixed set served in the teacher's order, and finishing marks the assignment as handed in.	`services/practice.ts`

What happens when an answer is submitted

1. Check the session belongs to this student and that the question is the current one.
2. Compute correct or not, and the new level and streak.
3. **One database transaction:** claim the question (a conditional update), save the attempt, update the mastery row, and, if this was the last question of an assignment, mark it handed in.
4. Return the verdict, the explanation and the new level.

> **Double-click safe.** Step 3's conditional update only succeeds once. Two simultaneous submissions of the same question produce one saved attempt and one polite "already answered" response. There's a test for exactly this.

What a teacher does

A teacher's loop: create a class, give students the code, assign work, then use the dashboard to see who needs help.

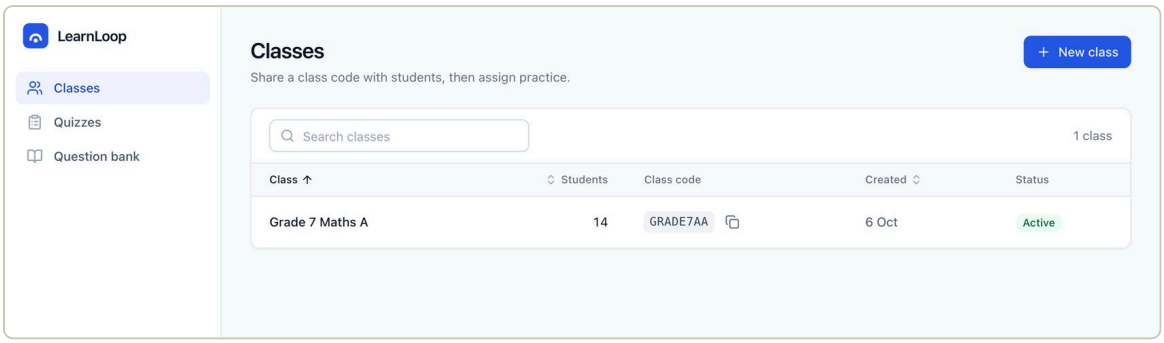


Fig 7 · Classes. Each has a random 8-character code from letters and digits that are hard to confuse (no 0, O, 1 or I). The table can be searched and sorted, and codes copy with one click.

The workflow

1. **Create a class.** A join code is generated. It can be rotated at any time (the old code stops working), and classes can be archived.
2. **Build the question bank.** Questions have a topic, a level from 1 to 5, a type (numeric or multiple choice), an answer key and a worked explanation. Each is `draft`, `approved` or `retired`; only approved ones reach students. In Week 3, AI-written questions will always arrive as drafts.
3. **Make a quiz** from approved questions, in the order clicked.
4. **Assign it** to the whole class or chosen students, with a future due date. Whole-class assignments also reach students who join later.
5. **Watch the dashboard**, which has an Overview, a sortable Students list and an Assignments tab. Open any student for details.

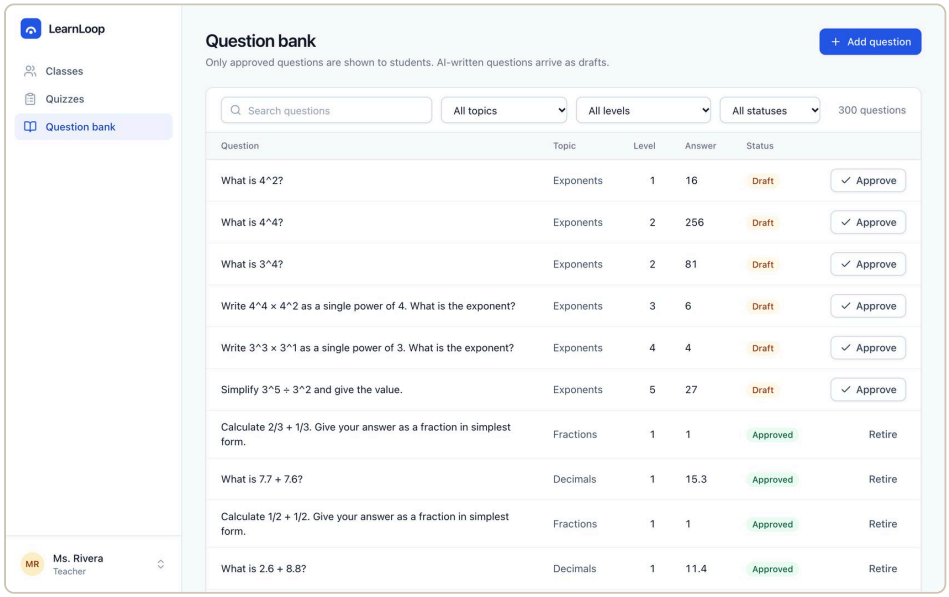


Fig 8 · The question bank: search, filter by topic, level and status, and approve or retire questions.

Assigning, and the dashboard

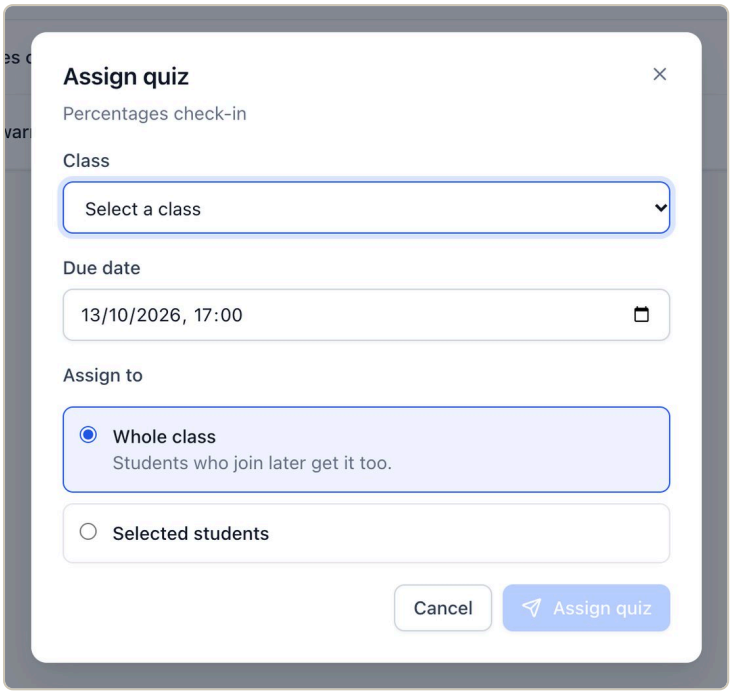


Fig 9 · Assigning a quiz.

Assigning

Pick the class, a due date and who gets it. Choosing "selected students" only allows students actually in that class. Past due dates and archived classes are refused.

Each chosen student gets an assignment slot. When they finish the quiz the slot is marked handed in, and that drives the completion figures below.

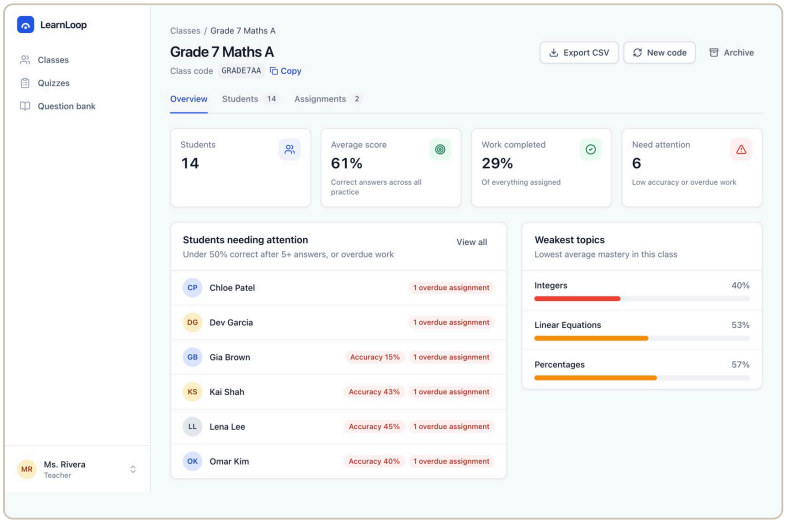


Fig 10 · The class dashboard (Overview tab).

Reading the dashboard

Average score	Average accuracy of students who have answered at least once.
Work completed	Finished assignment slots divided by all slots.
Need attention	At least 5 answers with accuracy under 50%, or any overdue assignment. Reasons are shown by the name.
Weakest topics	The three topics with the lowest average mastery.
Export CSV	Downloads the student table. Cells starting with `=` `+` `-` `@` are escaped so a bad name can't run as a spreadsheet formula.

Ownership. A teacher can only open their own classes and the students in them. Changing an ID in the URL to another teacher's class returns 403, and a test proves it.

One student, in detail

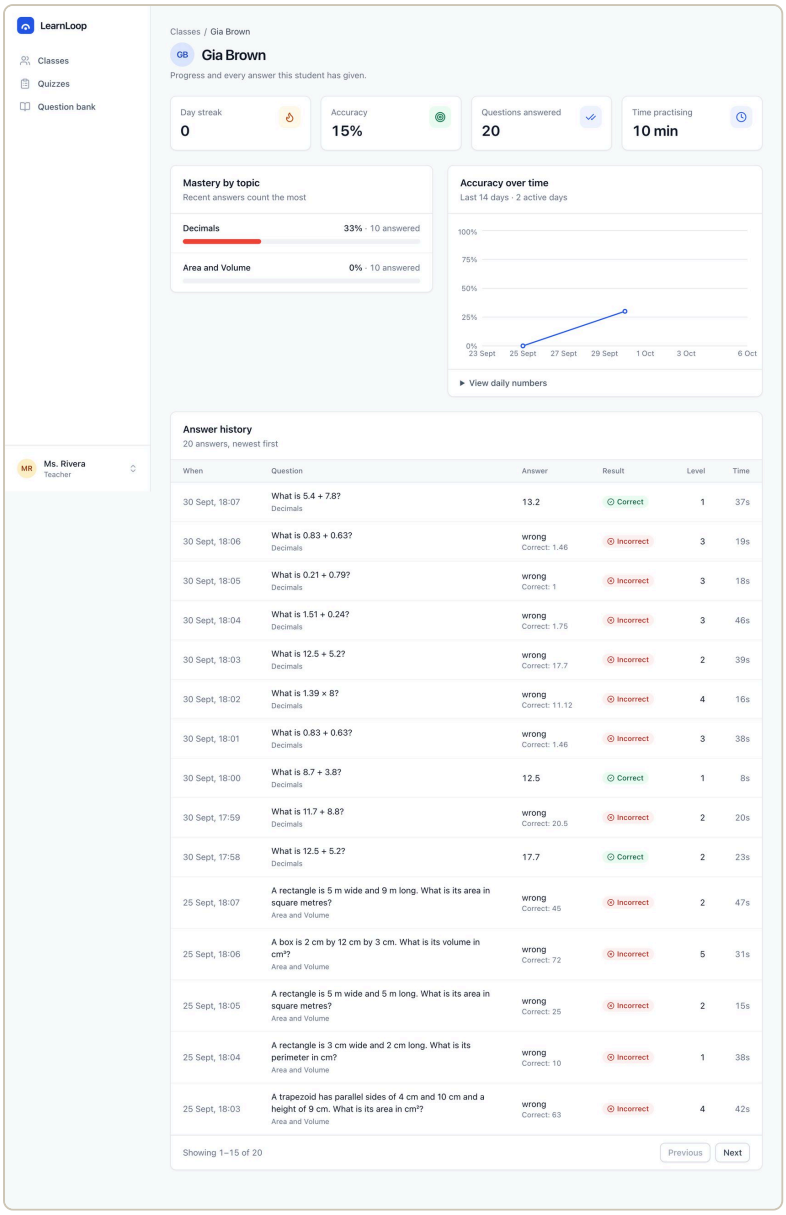


Fig 11 · Student detail.

Clicking a name from the dashboard opens the same progress view the student sees: streak, accuracy, mastery per topic and the 14-day trend.

Below it is **every answer** the student has given, newest first: when, the question, what they typed, whether it was right or wrong, the level, and the time taken. For wrong answers the key is shown too, so a teacher can see *what* mistake was made, not just that one was.

This is the page to open before a parent meeting or a revision lesson.

> **Access.** A teacher can open a student only if that student is enrolled in one of their classes. Any other student returns 403.

Users and the audit log

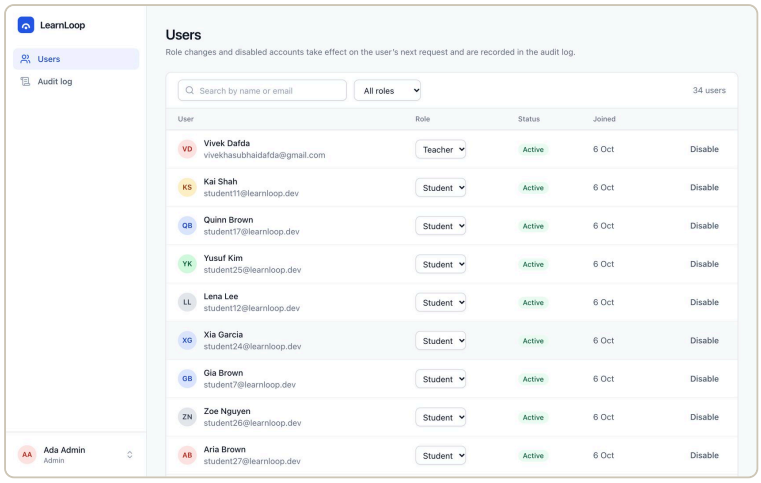


Fig 12 · Users: search, change a role, disable or enable an account. Admins cannot change themselves, so nobody can lock the system out.

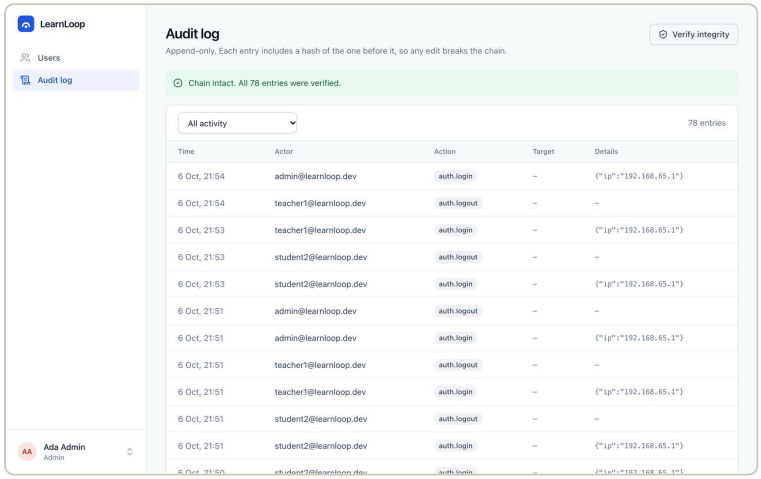
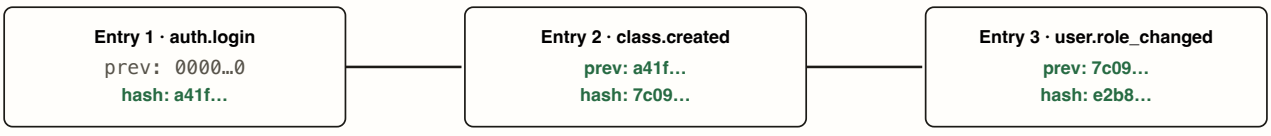


Fig 13 · The audit log, after pressing "Verify integrity".

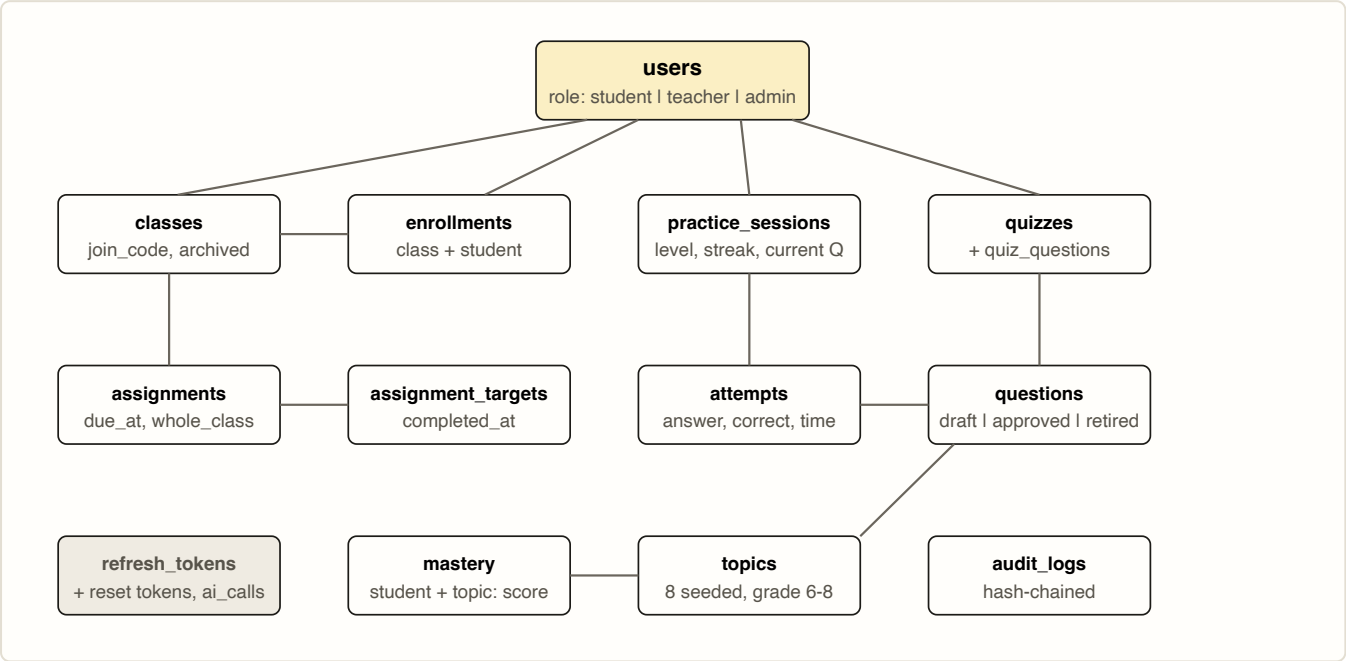
How the audit log can't be quietly changed



- Each entry stores a hash of its own content *and* the previous entry's hash. Change any row and every hash after it stops matching.
- "Verify integrity" re-computes all hashes and reports the first broken entry.
- A second defence: a Postgres **trigger** rejects any UPDATE or DELETE on the table, even from a database tool. Tests cover both the trigger and the detection if the trigger is bypassed.
- Recorded: logins and failures, token-reuse alarms, role changes, disabling, class, question, quiz and assignment actions. Raw passwords and tokens are never logged.

Where the data lives

PostgreSQL 16, accessed through Prisma. 16 tables. You can open it in DBeaver at `localhost:5432` (user, password and database are all `learnloop` in development).



Why it is shaped this way

- **Relationships are enforced by the database** (foreign keys, unique pairs), not just by code. A student can't be enrolled twice in a class; a question can't point to a missing topic.
- **Every lookup column has an index**, such as `(student_id, created_at)` on attempts.
- **Session state is in the table** (`practice_sessions`), not in server memory, so a restart or a second API server loses nothing.
- **Mastery is stored**, not recalculated from every attempt each time.
- `attempts` **and** `audit_logs` **only grow**. They are append-only and time-indexed, so they can be partitioned or archived later.
- **Demo data:** 1 admin, 2 teachers, 30 students and 300 computer-generated questions with worked answers, plus two weeks of practice history. All synthetic.

Redis holds only temporary counters (login rate limits). If it were wiped, nothing important would be lost.

Protection, layer by layer

Security was built in from week 1. Each layer assumes the one before it might fail.

LAYER	CONTROL	STOPS
Network	Strict security headers (CSP, HSTS, no framing, no sniffing), CORS only for known origins, 100 KB body limit	Script injection, clickjacking, oversized requests
Sessions	15-minute access tokens in memory; rotating refresh tokens in HttpOnly SameSite=Strict cookies; double-submit CSRF token	Token theft, forged cross-site requests
Passwords	Argon2id, 12-character minimum, same response for unknown email and wrong password	Cracking after a leak, account discovery
Abuse	Rate limits per IP and per account on login, signup, reset and join codes (Redis)	Brute-force and code guessing
Access	Role check on every route, plus an ownership check on every route with an ID; the user is re-read from the database on every request	Students reading each other's data, stale permissions
Input	Zod schemas on body, params and query; unknown fields rejected; Prisma only (no hand-built SQL); React escaping	Injection of all kinds
Errors	One safe shape: `{ code, message, requestId }`	Leaking stack traces, SQL or secrets
Records	Hash-chained audit log with a database trigger	Quietly rewriting history
Supply chain	Dependency audit, CodeQL, secret scanning, Docker image scanning, Dependabot; non-root containers	Vulnerable packages, leaked keys

> **The most common web bug is a student changing an ID in the address bar to see someone else's data.** Here every ID route goes through `services/access.ts`, and the tests try exactly that attack for students and teachers and expect a 403.

The full control-by-control list, with the OWASP Top 10 mapping and the threat model, is in `SECURITY.md`.

How we know it works

SUITE	TESTS	WHAT IT COVERS
Shared	15	Input rules, number parsing
API	128	Rules, endpoints, security, a real Postgres database
Web	11	Components, tabs, sorting, toasts, login flow

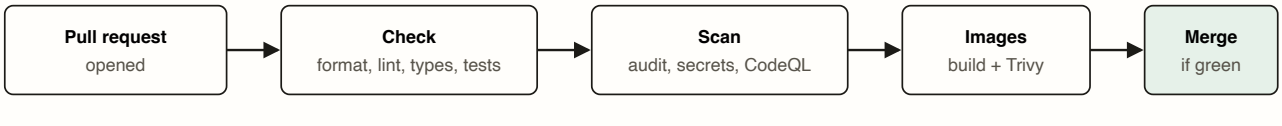
> **All 154 pass**, with lint, type-check, formatting and the production build also clean.

API line coverage is 85%; the requirement is 80%, and the build fails below it.

Tests worth knowing about

- **The RBAC matrix.** A test reads the list of all routes and, for each one, checks a logged-out call gets 401 and every wrong role gets 403. A new endpoint is covered automatically.
- **Ownership attacks.** Student A tries to read student B's progress, answer in B's session, or start B's assignment. Teacher A tries teacher B's class and quizzes. All must fail.
- **Token theft.** Replaying an old refresh token must revoke the family.
- **Tampering.** Editing the audit log must be blocked, and detected if the block is bypassed.
- **Concurrency.** Two simultaneous answers produce exactly one attempt.

The pipeline (GitHub Actions)



High or critical security findings fail the build. End-to-end browser tests, accessibility scans and a load test arrive in week 4.

Run it yourself

With Docker (everything at once)

`docker compose -f infra/docker-compose.yml up --build` then open `http://localhost:8080`. The API applies the database migrations and loads the demo data the first time it starts.

For development (changes appear instantly)

`npm install`, copy `.env.example` to `.env`, start only the databases with `docker compose -f infra/docker-compose.yml up -d postgres redis`, then `npm run db:migrate && npm run db:seed && npm run dev`. The web app is on port 5173 and the API on 4000.

Demo accounts

ROLE	EMAIL	NOTES
Student	`student2@learnloop.dev`	In Grade 7 Maths A, with history
Student	`student1@learnloop.dev`	Not in a class: try code `GRADE7AA`
Teacher	`teacher1@learnloop.dev`	Owns Grade 7 Maths A
Admin	`admin@learnloop.dev`	Users and audit log

Password for all: `LearnLoop!2026`

Useful commands

`npm test`	All 154 tests (needs the database running)
`npm run lint && npm run typecheck`	Code quality checks
`npx tsx prisma/seed.ts --reset`	Restore the demo data (run in apps/api)

The design system

The interface follows the conventions of real product UIs rather than a decorative theme. After two earlier attempts that still looked auto-generated, the rules were written down in `docs/design.md`.

- **App shell:** a sidebar with icons and a user menu, which becomes a top bar and drawer on phones.
- **One brand colour:** a single blue for actions and the active page, on neutral grays.
- **Colour means state:** green is correct or done, red is wrong or at risk, amber is pending.
- **Familiar patterns:** tabs, searchable and sortable tables, toasts, empty states, and a lesson page with a fixed bottom bar.
- **Plain type:** the system font, with tabular numbers for figures.
- **Accessible:** contrast of at least 4.5:1, keyboard friendly, no sideways scrolling from 360 px up.

Built to grow, and honest about the limits

The project rule is to design everything so it still works with far more users: 100,000 or more students, and a whole school logging in at 9:00.

Already in place

- **Stateless API.** Sessions are tokens plus database rows, so more API servers can run behind a load balancer.
- **Shared counters in Redis**, so rate limits stay correct across servers.
- **Pagination** on the main lists, and **indexes** on lookup columns.
- **Safe retries:** conditional updates and unique constraints make double clicks and repeated requests harmless.
- **Hard limits** on page size, body size and login attempts.
- **Readiness checks** (`/ready`) that test the database and Redis.

Known bottlenecks (to fix before real scale)

WHERE	PROBLEM AT SCALE	PLANNED FIX
Audit log writes	One lock serialises every audited action (each login)	Shard the chain, or write through a queue
Every request	One database read to re-check role and status	Cache in Redis for about a minute
Choosing the next question	Random pick scans more rows as the bank grows	Indexed random key or per-level pools
Class dashboard	Recomputed from scratch on each load	Cache per class, or keep running counters
Big history tables	Page-number paging gets slow deep in the list	Cursor-based paging
Container start-up	Each replica runs migrations and seeding	One migration job at deploy time

What's next

Week 3: AI and the MCP server

- AI explanations of mistakes and hints that never reveal the answer
- Teachers can ask AI for draft questions; drafts need approval
- Personal data removed before any text goes to a model; defence against prompt injection
- An MCP server with 6 tools so an AI assistant can use the platform, with the same permissions as the user
- Free Gemini model by default, queued and cached to stay within limits

Week 4: hardening and release

- Browser end-to-end and accessibility tests
- A load test against the 300 ms target
- Deploy: front end on Vercel, API and database on Render
- Demo video and release automation